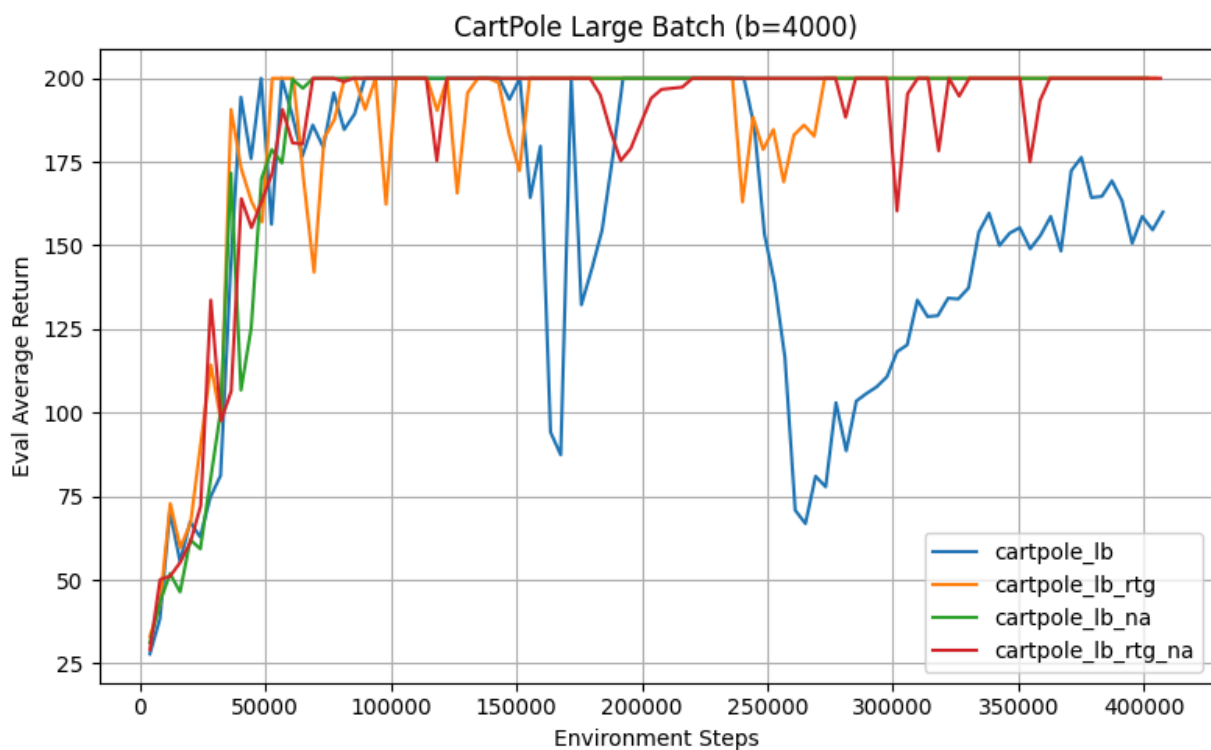
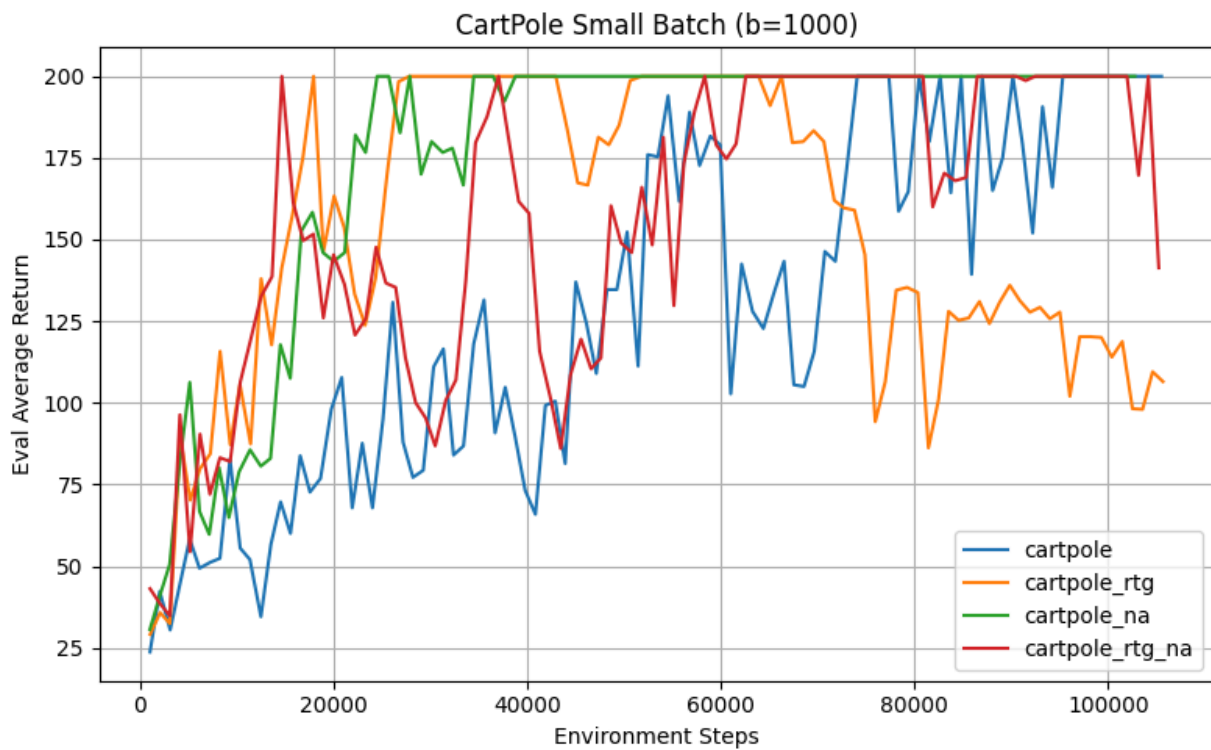


3. CartPole



- Which value estimator has better performance without advantage normalization: trajectory-centric or reward-to-go?

Reward-to-go (-rtg) generally performs better. It's a lower-variance estimator because it only uses rewards after timestep t , excluding irrelevant past rewards from the Q estimate. Hence, we operate with smaller numbers leading to lower variances

- Between the two value estimators, why do you think one is generally preferred over the other?

Reward-to-go is generally preferred because the trajectory-centric estimate includes rewards that couldn't have been caused by that action at the timestep. RTG removes this causal inconsistency and reduces variance, while still having the same variance (unbiased estimator).

- Did advantage normalization help?

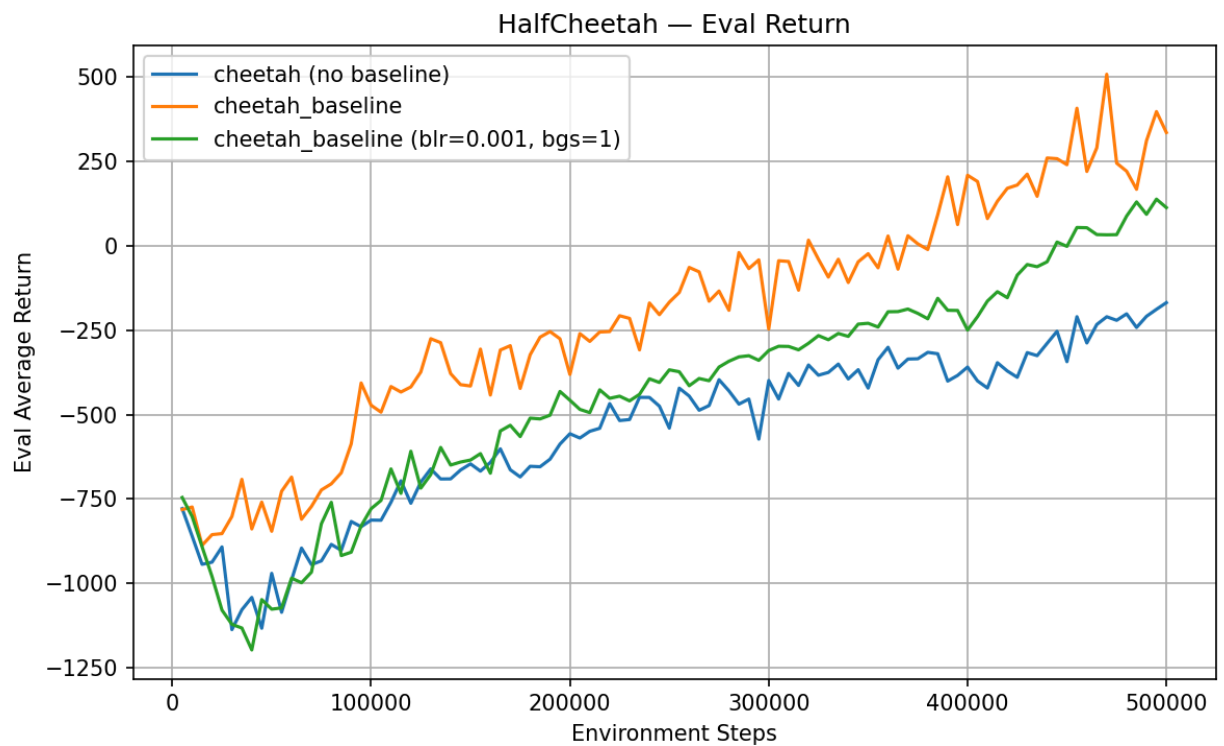
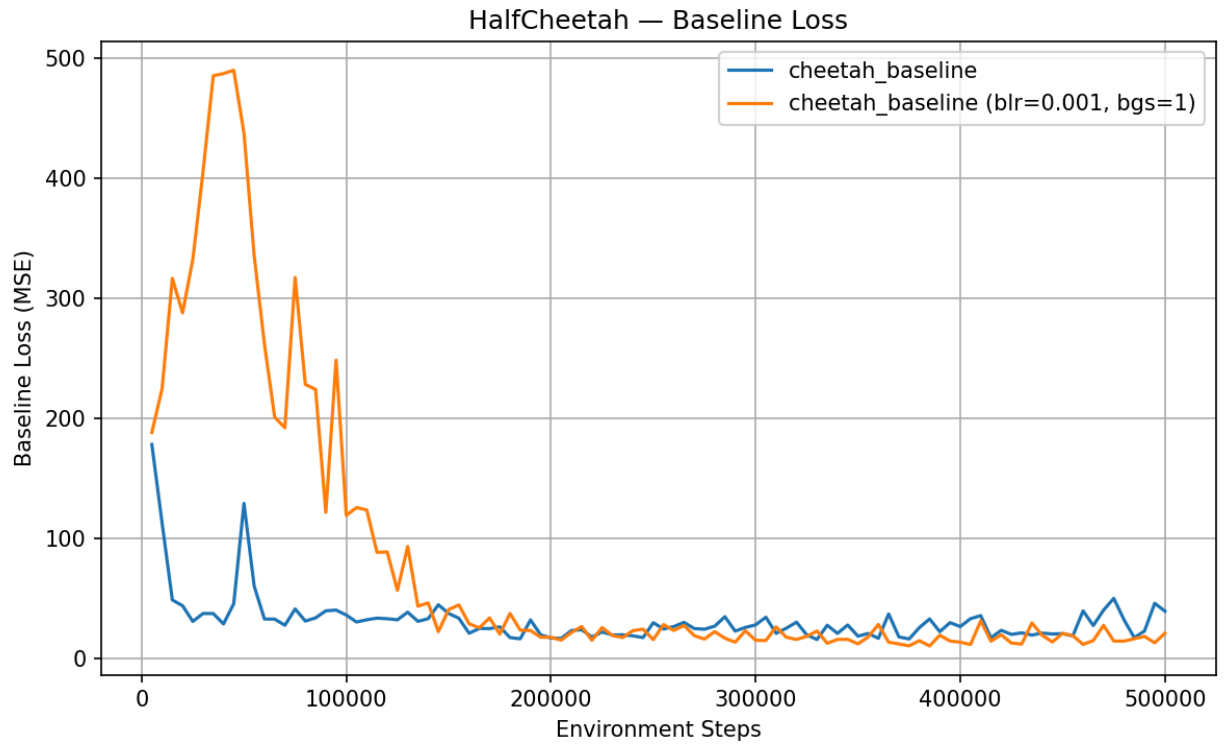
Yes, advantage normalization helped improve convergence speed and stability.

- Did the batch size make an impact?

Yes, larger batches (4000) generally produce smoother, more stable learning curves because gradient estimates are less noisy.

```
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 --exp_name cartpole
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg --exp_name cartpole_rtg
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -na --exp_name cartpole_na
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg -na --exp_name
cartpole_rtg_na
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 --exp_name cartpole_lb
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg --exp_name
cartpole_lb_rtg
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -na --exp_name
cartpole_lb_na
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg -na --exp_name
cartpole_lb_rtg_na
```

4. HalfCheetah



- Run another experiment with a decreased number of baseline gradient steps (-bgs) or baseline learning rate (-blr). How does this affect (a) the baseline learning curve and (b) the performance of the policy?

The weaker baseline (blr=0.001, bgs=1) shows a higher learning curve and more slowly decreasing MSE loss than regular baseline. The value network gets fewer/smaller gradient updates per iteration, so it tracks the true Q-values less accurately.

The policy performs worse (lower reward) than the default baseline, because a poorly-fit value function provides a noisy baseline. It may not subtract the right amount from the Q-values, leaving more variance in the advantage estimates.

No baseline (default hyperparams)

```
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --exp_name cheetah
```

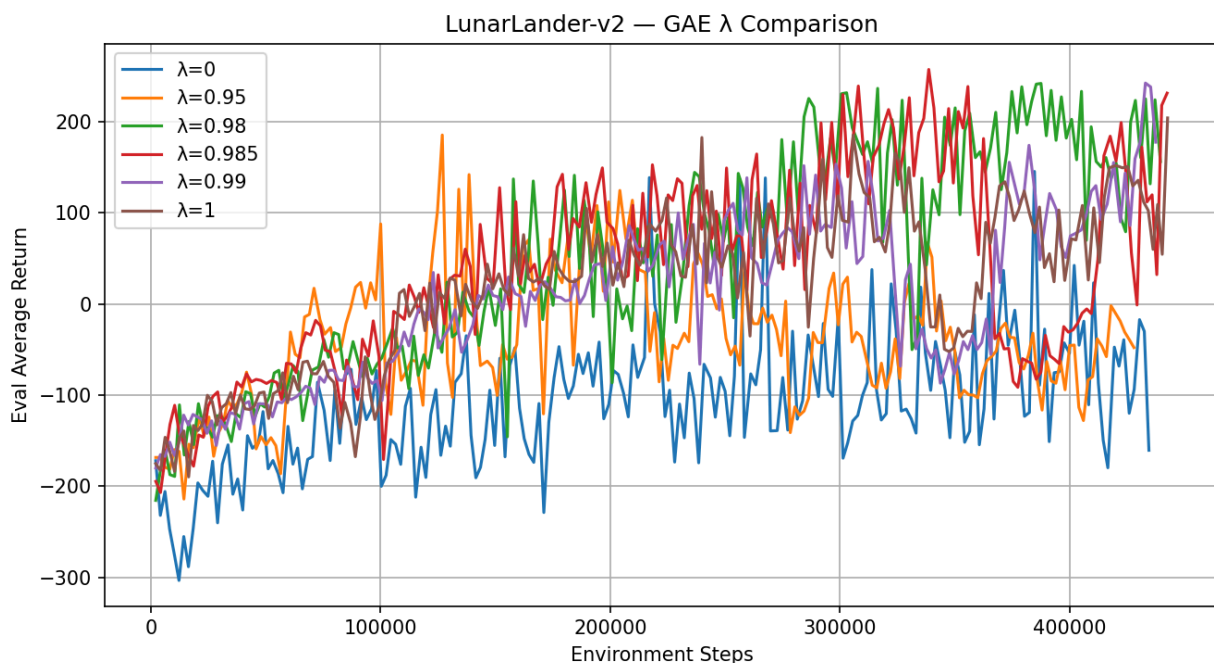
Baseline (default hyperparams)

```
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 5 --exp_name cheetah_baseline
```

Weaker baseline (decreased blr and bgs from defaults)

```
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.001 -bgs 1 --exp_name cheetah_baseline_small_bgs_blr
```

5. LunarLander



- Consider the parameter λ . What does $\lambda = 0$ correspond to? What about $\lambda = 1$? Relate this to the task performance in LunarLander-v2 in one or two sentences.

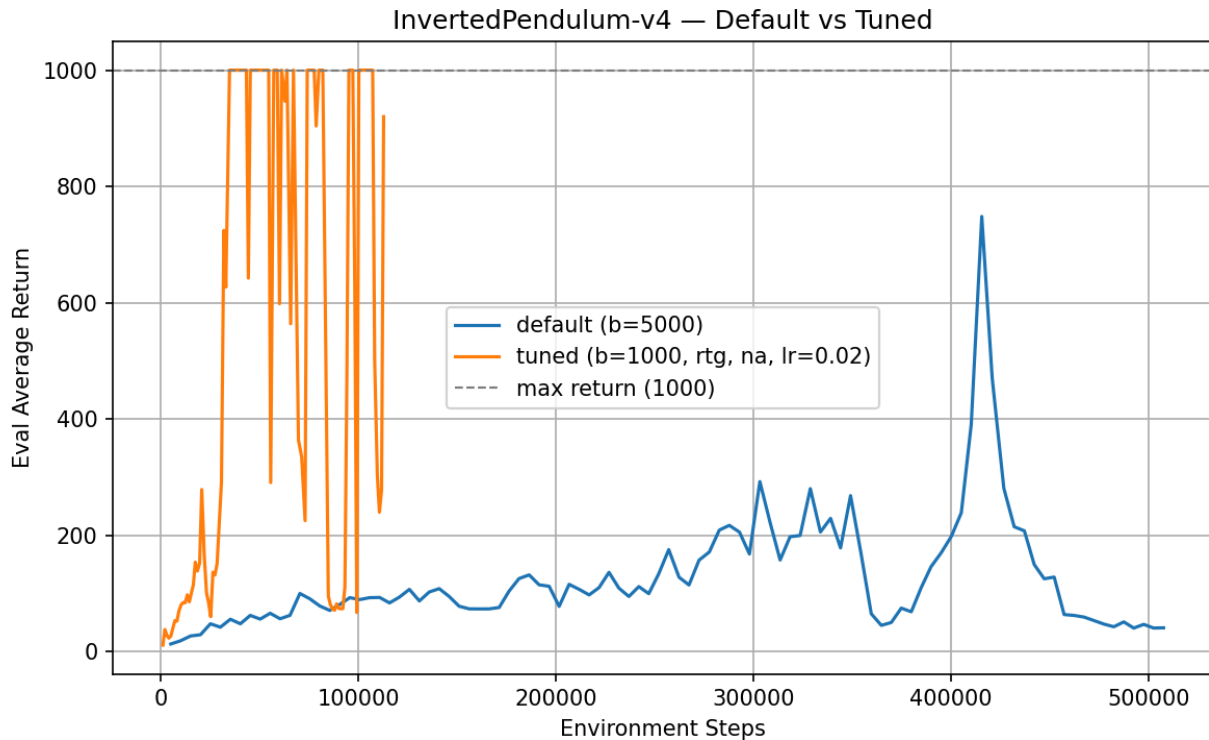
$\lambda=0$: GAE reduces to the 1-step TD advantage. This is low variance but high bias, since it relies entirely on the (imperfect) value function bootstrap.

$\lambda=1$: GAE reduces to the full reward-to-go minus the baseline. This is unbiased but high variance, since it sums all future rewards without any bootstrapping.

In experiments, we see that a midway λ value, like $\lambda=0.98$ performs the best with the most stable training and highest average reward. Intermediate values of λ (0.95–0.99) typically outperform both extremes by balancing bias and variance. $\lambda=0$ converges slowly or to a suboptimal policy due to heavy reliance on an early inaccurate value function, while $\lambda=1$ struggles with high-variance gradient estimates.

```
uv run src/scripts/run.py --env_name LunarLander-v2 -n 300 -b 2000 -eb 1000 -rtg
--use_baseline --discount 0.99 -lr 0.001 --gae_lambda 0 --exp_name lunar_lander_lambda0
uv run src/scripts/run.py --env_name LunarLander-v2 -n 300 -b 2000 -eb 1000 -rtg
--use_baseline --discount 0.99 -lr 0.001 --gae_lambda 0.95 --exp_name
lunar_lander_lambda0.95
uv run src/scripts/run.py --env_name LunarLander-v2 -n 300 -b 2000 -eb 1000 -rtg
--use_baseline --discount 0.99 -lr 0.001 --gae_lambda 0.98 --exp_name
lunar_lander_lambda0.98
uv run src/scripts/run.py --env_name LunarLander-v2 -n 300 -b 2000 -eb 1000 -rtg
--use_baseline --discount 0.99 -lr 0.001 --gae_lambda 0.985 --exp_name
lunar_lander_lambda0.985
uv run src/scripts/run.py --env_name LunarLander-v2 -n 300 -b 2000 -eb 1000 -rtg
--use_baseline --discount 0.99 -lr 0.001 --gae_lambda 0.99 --exp_name
lunar_lander_lambda0.99
uv run src/scripts/run.py --env_name LunarLander-v2 -n 300 -b 2000 -eb 1000 -rtg
--use_baseline --discount 0.99 -lr 0.001 --gae_lambda 1 --exp_name lunar_lander_lambda1
```

6. Inverted Pendulum



- Provide the best set of hyperparameters on InvertedPendulum-v4 and the exact command line configuration. Briefly discuss which hyperparameters mattered in your tuning process.

Batch size reduction from 5000 to 1000. The default uses 5000 steps per iteration, meaning 100K steps = only 20 policy gradient updates. Reducing to 1000 gives 100 updates within the same budget, which is the primary reason for faster convergence.

Reward-to-go reduces gradient variance by using only future rewards, giving faster and more stable convergence due to a lower variance estimator of the gradient.

Advantage normalization is also a free upgrade for faster and more stable convergence.

Increasing the learning rate from 0.005 to 0.02 because the higher rate takes larger steps per update which could lead to faster convergence.

```
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 1000 -eb 1000 -rtg -na -lr 0.02 --exp_name pendulum_b1000_rtg_na
```